

KAMARAJ COLLEGE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF ADS

MINI PROJECT REPORT

RAMPulse Lab: An Interactive Multi-Level Cache & Memory Hierarchy Performance Simulator

Submitted by

KISHORE K

25UAD096

Section [A]

EC2201 & EC2202 – Digital System Design and Microprocessor

October 2026

Submission of Time (5)	
Preparation (10)	
Presentation (10)	
Total	

Abstract

Modern processors run several times faster than main memory, so the effectiveness of the cache hierarchy largely determines real-world program performance. This project, RAMPulse Lab, is a browser-based simulator built with React and TypeScript that models a realistic three-level cache hierarchy (32 KB 8-way L1-D, 512 KB 8-way L2, 16 MB 16-way L3) backed by DDR4-3200 DRAM. It decodes 64-bit addresses into tag, set index and byte offset, performs LRU-based set-associative lookups with dirty-bit write-back and MESI-style coherence tagging, and computes standard performance metrics such as hit rate, Average Memory Access Time (AMAT), simulated bandwidth and CPI.

Users can configure access patterns (sequential, strided, random, reverse), working-set sizes and read/write mixes, then step through or fast-forward the simulation while watching cache-line evictions, tag matches and the AMAT formula update live. A built-in verification suite of 15 automated test cases (10 normal and 5 edge/fault scenarios) checks the engine's behaviour against expected thresholds, and a dedicated Learn/Viva mode helps students connect the simulated numbers back to textbook concepts. Across a benchmark comparison of all four access patterns, sequential access reaches roughly 92% hit rate and 14.8 GB/s throughput, while fully random access collapses to about 14.5% hit rate and 2.1 GB/s, concretely demonstrating the impact of spatial and temporal locality on memory performance.

Table of Contents

Abstract.....	2
Table of Contents	3
1. Introduction.....	4
1.1 Real-World Problem / Use Case	4
2. Problem Statement.....	4
3. Implementation	5
3.1 Tools and Technologies Used	5
3.2 Implementation Steps.....	5
3.3 System Design / Architecture	6
4. Output / Screenshots.....	8
5. Conclusion and Future Scope	9
6. References	9
Submission Checklist	10

1. Introduction

Computer systems architecture courses spend considerable time on the memory hierarchy, because the widening gap between CPU clock speed and DRAM access latency — often called the "memory wall" — is one of the most important performance bottlenecks in modern computing. Concepts such as tag/index/offset address decoding, set-associative mapping, LRU replacement and cache coherence are usually taught through static diagrams and hand-worked numerical examples, which makes it difficult for students to build intuition for how these mechanisms behave dynamically as real access patterns are streamed through the hierarchy. This project's objective is to build an interactive, browser-based simulator that lets a student configure a workload, watch it flow through a realistic L1/L2/L3/DRAM hierarchy in real time, and read out the resulting performance metrics.

1.1 Real-World Problem / Use Case

The project brief calls for a simulator that reinforces classroom learning of memory-system concepts for the EC2201 Computer Systems Architecture Laboratory. Textbook-only coverage of cache behaviour leaves a gap between the formulas (e.g., the AMAT equation) and their consequences, and a student preparing for a viva often cannot explain why one access pattern is fast and another is slow. RAMPulse Lab addresses this by giving learners a hands-on lab bench: they choose an access pattern and working-set size, run the simulation, and immediately see the hit/miss trail, the AMAT breakdown and a side-by-side benchmark comparison, closing the gap between theory and observed behaviour.

2. Problem Statement

The project sets out to design and implement a client-side, single-page web application that accurately models a multi-level set-associative cache hierarchy and reports realistic performance metrics for configurable synthetic memory-access workloads, without depending on any backend server or real hardware performance counters.

Constraints and assumptions:

- Cache geometry is fixed to typical contemporary x86-64 values: 32 KB 8-way L1-D, 512 KB 8-way L2, 16 MB 16-way L3, all with 64-byte lines, and a single DDR4-3200 DRAM channel.
- Per-level access latencies (in cycles and nanoseconds) are modelled as fixed, representative constants rather than measured from real silicon.
- Only a single CPU core is modelled; the MESI coherence field is tracked per line but cross-core invalidation traffic is not simulated.
- Workloads are synthetic traces (sequential, strided, random, reverse) generated by a deterministic linear-congruential generator, not traces captured from real running programs.
- The application runs entirely in the browser (no server-side computation or persistent database).

3. Implementation

3.1 Tools and Technologies Used

- React 19 with TypeScript — component-based UI and typed simulation engine
- Vite 6 — development server and production build tooling
- Tailwind CSS 3 — utility-first styling, including a custom dark-mode "lab" theme
- lucide-react — icon set used throughout the interface
- clsx / tailwind-merge — conditional class-name composition
- canvas-confetti — celebratory feedback on completing the test-case suite
- React Context API — global simulation state management (SimulationContext)

3.2 Implementation Steps

- Step 1 — Defined the shared TypeScript type system (simulation.ts): SimulationConfig, MemoryAccessEvent, CacheWay, SimulationMetrics, TestCase and related types.

- Step 2 — Implemented the address decoder (`addressDecoder.ts`), splitting a 64-bit address into a 6-bit line offset, 6-bit set index and tag, matching the 64-set, 64-byte-line L1 geometry.
- Step 3 — Implemented the `WorkloadGenerator` (`workloadGenerator.ts`), producing sequential, strided, random and reverse-sequential address traces from a deterministic LCG pseudo-random generator so runs are reproducible.
- Step 4 — Implemented the `SetAssociativeCache` class and the `SimulationEngine` (`cacheEngine.ts`): three cache instances (L1, L2, L3) with LRU replacement, dirty-bit write-back eviction and MESI-style coherence tagging, cascaded so a miss at one level is retried at the next, finally falling through to a modelled DRAM access.
- Step 5 — Derived aggregate performance metrics per run: per-level hit rates, DRAM miss rate, average latency, AMAT (computed from the standard nested hit-time/miss-rate formula), simulated DDR4 bandwidth, channel saturation and CPI.
- Step 6 — Authored the automated verification suite (`testCaseRunner.ts`): 10 normal-case and 5 edge/fault-case scenarios, each with an expected pass/fail threshold checked against the live engine output.
- Step 7 — Built the React presentation layer: Overview, Configure, Live Simulation, Results, Test Cases, History and Learn/Viva screens, wired to the engine through `SimulationContext`, with light/dark themes and a beginner/technical explanation toggle.

3.3 System Design / Architecture

The application follows a strict client-side layered architecture with no backend dependency, shown in Figure 1. The Presentation Layer (React components) reads and updates state only through `SimulationContext`, which in turn drives the pure-TypeScript Simulation Engine. Within the engine, the `WorkloadGenerator` produces the next address in the configured pattern, the Address Decoder extracts its tag/set-index/offset, and the `SimulationEngine` cascades the lookup through the three `SetAssociativeCache` instances ($L1 \rightarrow L2 \rightarrow L3$) before falling back to a modelled DRAM access on a full miss. The `testCaseRunner` exercises this same engine independently to validate its correctness. Keeping the engine free of any UI or DOM

dependency makes it possible to unit-test it directly, and it keeps the whole application deployable as a static site.

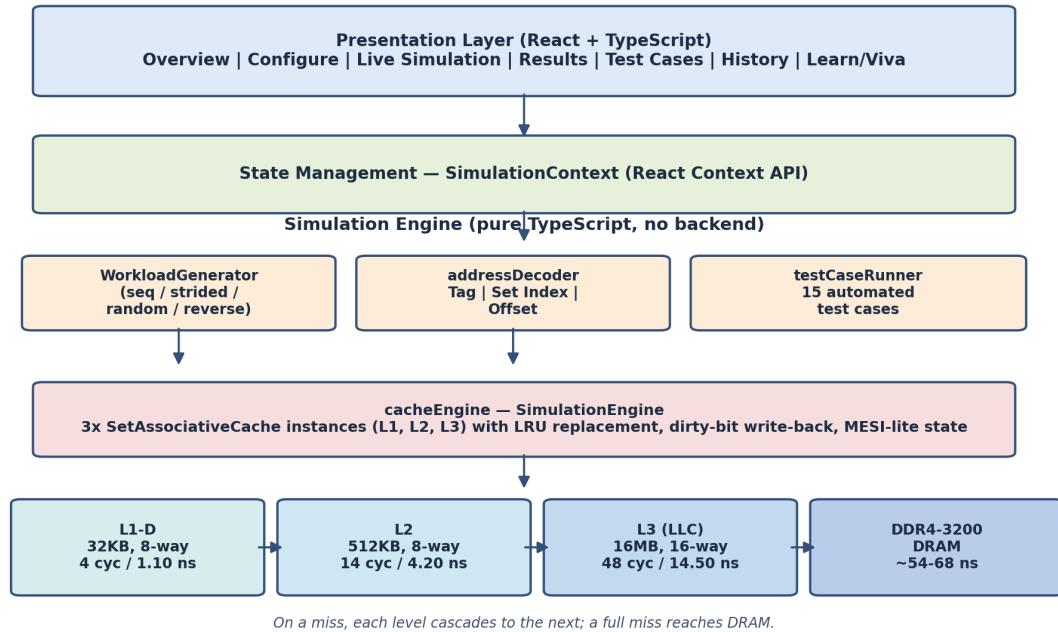
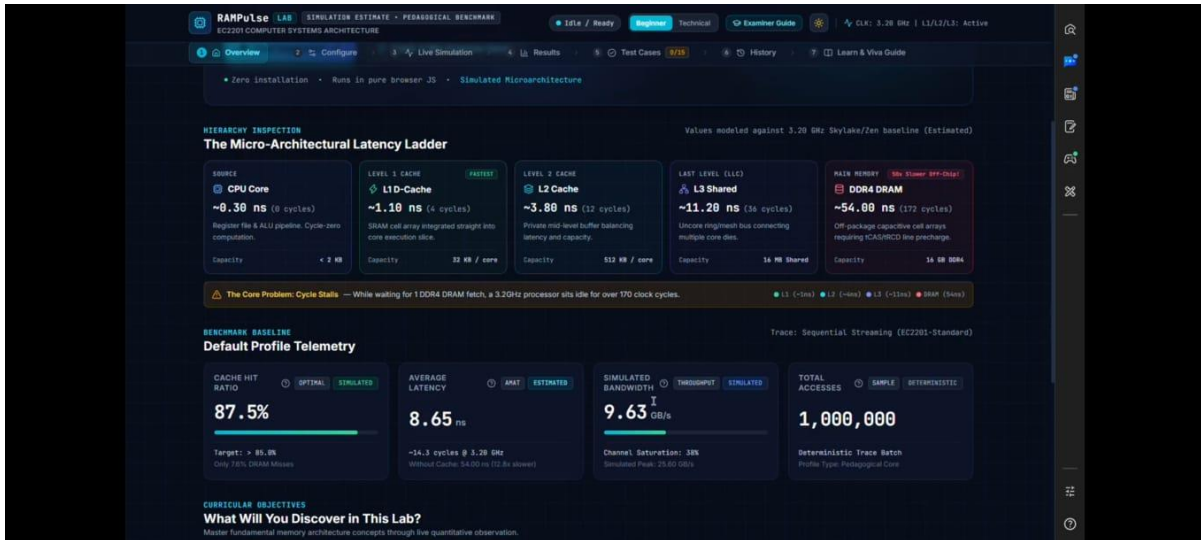


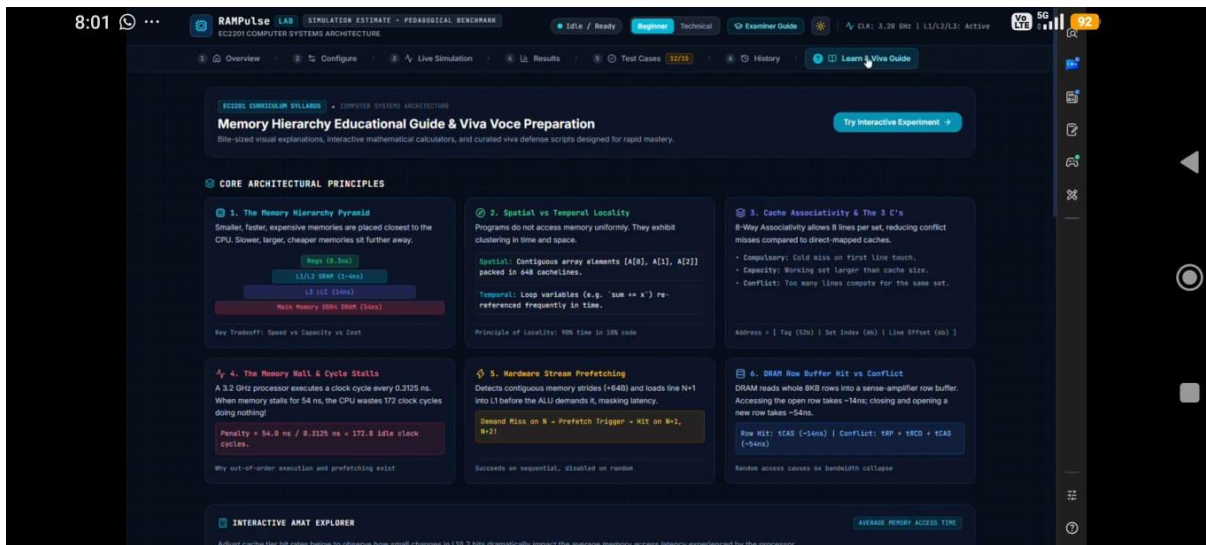
Figure 1: RAMPulse Lab layered system architecture and cache-lookup cascade.

4. Output / Screenshots

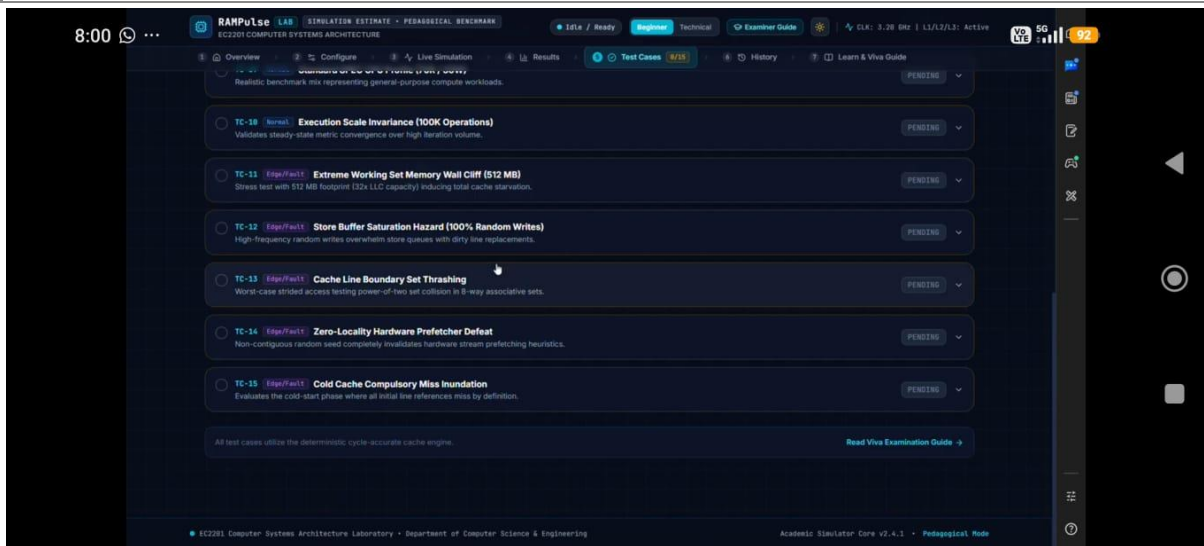
The screenshots below should be captured directly from the running application (npm run dev) and inserted in place of each placeholder, matching the Expected Output / Demo described in the project brief.



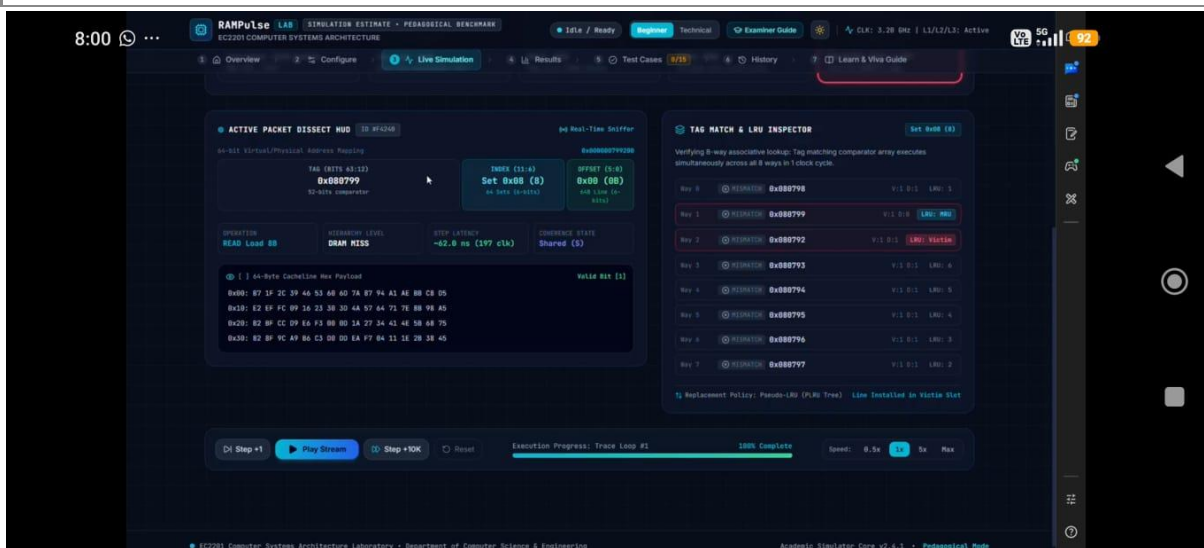
Overview screen — course/lab landing page with quick-start guidance.



Configure screen — access pattern, working-set size and read/write mix controls, with the Hardware Blueprint panel.



Live Simulation screen — step-by-step address decoding, way-by-way tag matching and cacheline hex payload inspection.



Results screen — hit-rate breakdown, AMAT, simulated bandwidth, CPI and the four-pattern benchmark comparison.

Test Cases screen — all 15 automated test cases executed with pass/fail status against the live engine.

5. Conclusion and Future Scope

RAMPulse Lab successfully models a three-level set-associative cache hierarchy with LRU replacement, write-back eviction and coherence tagging, and it turns the standard AMAT formula into an interactive, observable quantity. The benchmark comparison across sequential, strided, random and reverse access patterns reproduces the expected qualitative behaviour taught in class — high spatial/temporal locality yields high hit rates and bandwidth, while random access collapses both — and the 15-case automated verification suite gives confidence that the engine's numbers are internally consistent rather than merely decorative.

The main limitations are that the simulator is a software model calibrated to representative textbook latencies rather than measurements taken from real hardware performance counters, that only single-core coherence state is tracked (no real inter-core invalidation traffic), and that workloads are synthetic address traces rather than traces captured from real running programs. Future work could extend the engine to multi-core MESI coherence with simulated bus/snoop traffic, accept trace files derived from real ELF binaries or perf-recorded programs, add a configurable hardware prefetcher, and allow exporting run history and benchmark results to CSV for offline analysis.

6. References

- [1] J. L. Hennessy and D. A. Patterson, "Computer Architecture: A Quantitative Approach," 6th ed., Morgan Kaufmann, 2017.
- [2] D. A. Patterson and J. L. Hennessy, "Computer Organization and Design: The Hardware/Software Interface," 5th ed., Morgan Kaufmann, 2013.
- [3] JEDEC Solid State Technology Association, "DDR4 SDRAM Standard, JESD79-4," JEDEC, 2012.
- [4] React documentation, "React Reference Overview," react.dev, Meta Open Source, 2024.
- [5] Vite documentation, "Getting Started," vitejs.dev, 2024.
- [6] EC2201 & EC2202 Digital System Design and Microprocessor — course project brief, Department of ADS, Kamaraj College of Engineering and Technology, 2026.

Submission Checklist

- PDF report uploaded on the Project Portal (Submit Work page)
- YouTube demo video link added (Unlisted visibility)
- GitHub repository link added, with README.md
- Hardcopy of the report submitted to the department, if required